CMPSC 448 - Midterm Project Report

Nicholas Zubarev

October 11, 2026

1. Introduction

Large Language Models (LLMs) such as ChatGPT, Claude, and Gemini are increasingly used to generate written content. Although these models often produce similar responses to the same questions, their writing styles, vocabulary, formatting, and sentence structures may contain patterns that distinguish them from one another. The goal of this project is to investigate whether machine learning classifiers can identify which LLM generated a particular response.

Specifically, I implemented and evaluated two neural network architectures: a Convolutional Neural Network (CNN) and a Long Short-Term Memory network (LSTM). The project focused on two research questions:

RQ1: Can we identify which LLM generated a response?

RQ2: Does including the original user prompt improve classification accuracy?

To answer these questions, I trained and evaluated both models using different combinations of user prompts and LLM-generated responses.

2. Dataset and Preparation

I used the publicly available TXD-22 dataset, which contains 66,000 question-answer records from 22 authorship categories. I selected responses from ChatGPT, Claude, and Gemini, with 3,000 examples from each model, giving me 9,000 examples total. The dataset contained the original prompt, the generated response, and the name of the LLM that produced it. These were

stored as LLM_Input, LLM_output, and LLM_name. I cleaned the dataset by checking for missing values and duplicates. I then split the data into training, validation, and testing sets. The split was done by a unique question so the same prompt would not appear in multiple sets.

Dataset	**Examples**
Training	7,198
Validation	898
Testing	904
Total	**9,000**

Before training, the text was converted to lowercase and tokenized. The vocabulary was limited to 12,000 tokens, and sequences were padded or truncated to a maximum length of 160 tokens. The dataset was kept balanced across the three LLMs to make the classification results easier to compare.

3. Model Implementation and Training

I implemented the CNN and LSTM models using Python and PyTorch. The CNN used a word embedding layer, convolutional layers with kernel sizes of 3, 4, and 5, max pooling, dropout, and a final classification layer. Its purpose was to identify short patterns in the text that could help distinguish the different LLMs. The LSTM used word embeddings and a bidirectional recurrent layer to process text sequences. Unlike the CNN, the LSTM was designed to learn patterns based on the order and relationships between words. Both models used the same general training settings:

Parameter	Value
Optimizer	Adam
Learning rate	0.001
Batch size	64
Epochs	5
Embedding size	96
Dropout	0.3
Loss function	Cross-entropy

Each model was trained using three input configurations: the user prompt only, the LLM response only, and the prompt combined with the response. This resulted in six experiments. I measured performance using accuracy and macro F1 score, and selected the best checkpoint using validation accuracy.

4. Results

Model	Input Used	Accuracy	Macro F1
CNN	Prompt only	33.3%	0.27
CNN	Response only	**88.94%**	**0.889**
CNN	Prompt + Response	87.5%	0.876
LSTM	Prompt only	33.3%	0.194
LSTM	Response only	80.09%	0.801
LSTM	Prompt + Response	75.44%	0.752

For **RQ1**, both models were able to identify which LLM generated a response with accuracy well above random guessing. The CNN achieved the best result at 88.94%, while the LSTM reached 80.09%.

For **RQ2**, the results showed that including the original prompt did not improve classification. When using prompts alone, both models achieved approximately 33.3% accuracy, which is equivalent to random guessing between three balanced categories.

Using the prompt and response together also produced slightly lower accuracy than using the response alone. Overall, the CNN performed better than the LSTM, and the response-only configuration produced the highest accuracy for both models.

5. Analysis and Discussion

The results suggest that different LLMs have recognizable patterns in their generated text. The CNN performed better than the LSTM, possibly because its convolutional layers were more effective at identifying short phrases, vocabulary patterns, and formatting differences. The CNN's best result was 88.94% accuracy, compared with 80.09% for the LSTM. This suggests that local text patterns were particularly useful for distinguishing between ChatGPT, Claude, and Gemini in this dataset. Looking at the CNN's output-only classification results, Claude was the easiest model to identify, with an F1 score of 0.933. ChatGPT had an F1 score of 0.863, while Gemini had an F1 score of 0.872. The most common classification mistake was predicting Gemini when the actual response came from ChatGPT. This happened 39 times, suggesting that some responses from these models may share similar writing patterns. Another important finding was that the prompt-only classifiers performed at chance level. This makes sense because many of the same questions were given to all three LLMs. The prompt itself did not provide useful information about which model generated the response. Adding the prompt to the response also reduced accuracy slightly. One possible explanation is that the extra prompt text introduced

information that was not useful for identifying the LLM. There were some limitations to this project. Only three LLM families were included, and both models were trained for five epochs. The input sequences were also limited to 160 tokens, which may have removed useful information from longer responses. Different datasets or training settings could produce different results.

6. Lessons Learned

This project helped me better understand how CNNs and LSTMs can be used for text classification. One important thing I learned was the importance of preparing the dataset correctly. Since multiple LLMs answered many of the same questions, separating the data by unique prompt was important to prevent data leakage between training and testing. I also learned that different neural network architectures can perform differently on the same task. Although both models were able to classify LLM responses, the CNN performed noticeably better than the LSTM. Another interesting result was that adding more information did not necessarily improve performance. Including the original user prompt actually lowered accuracy compared with using the response alone. I also gained experience training neural networks using PyTorch, evaluating models with accuracy and F1 scores, and comparing results across different experiments.

7. Conclusion

This project showed that machine learning can be used to identify which LLM generated a text response. Using 9,000 examples from ChatGPT, Claude, and Gemini, I trained CNN and LSTM

classifiers and evaluated their performance across three different input configurations. The CNN performed the best, achieving **88.94% accuracy** using only the generated response. The LSTM achieved 80.09% accuracy using the same configuration. The experiments also showed that including the original user prompt did not improve performance, and both prompt-only models performed close to random guessing. Overall, the results suggest that LLM-generated responses contain identifiable writing patterns and that CNNs can be effective at learning these differences.

8. References

1. TXD-22 Dataset: https://data.mendeley.com/datasets/prcjcggtjf/1

2. PyTorch Documentation: https://pytorch.org/docs/stable/

3. Scikit-learn Documentation: https://scikit-learn.org/stable/